

조건 필터링과 정렬

df[조건] · sort_values · copy

위치를 몰라도 조건만 알면 원하는 데이터를 다 찾아냅니다 — 현장 질문에 답하는 도구
13_02 조건 필터링과 정렬 · 전 범위 (실습 1~6)

핵심 키워드

Boolean Series · df[조건] · .sum() · (A) & (B) · (A) | (B) · ~(조건)
.isin([목록]) · .between(소, 대) · .str.contains()
.sort_values() · ascending=False · 다중 키 정렬 · 메서드 체이닝 · .copy() ·
SettingWithCopyWarning

2026년 8월 14일 (금) · 19일차 · 47~48차시

국립금오공과대학교 컴퓨터공학부 박민호

이 책을 읽는 법

뱃지 · 자료 고지 · 어제와의 연결

1. 항목 뱃지

오늘은 연산자와 메서드가 반씩 나옵니다. `&` · `|` · `~`는 기호 하나로 동작하는 연산자이고, `.isin()` · `.between()` · `.sort_values()`는 배열 뒤에 점을 찍고 부르는 메서드입니다.

뱃지	판별법	이번 회차 예
연산자	기호 하나로 동작	<code>>=</code> · <code>==</code> · <code>&</code> · <code> </code> · <code>~</code>
메서드	점 + 이름 + 소괄호	<code>.isin()</code> · <code>.between()</code> · <code>.sort_values()</code> · <code>.copy()</code>
문법	표기 규칙 자체	<code>df[조건]</code> · <code>(A) & (B)</code>
자료형	값의 종류	Boolean Series · bool
키워드	메서드 괄호 안의 이름=값	<code>ascending=</code> · <code>by=</code>
개념	문법이 아닌 사고방식	메서드 체이닝 · 워크플로우 5단계

2. 이번 회차 자료 고지

원자료에 어긋난 부분이 몇 군데 있어 먼저 밝혀 둡니다. 이 책은 모두 직접 실행해 확인한 숫자를 실습니다.

항목	상태	이 책에서
교안 13_02	있음 (54쪽 전 범위)	STEP 1~9의 뼈대
강사 코드	13_07~13_12만 (실습 1~4)	정렬 코드가 없어 교안 p34~p43 + 전사본으로 복원
전사본(STT)	있음 · 7조각 292분	중복 · 누락 없음 — 정렬 수업이 81~97% 구간에 온전히 있음
교안 예상 결과	실데이터와 불일치	부록 B-3에 대조표 수록
제출 코드	SyntaxError로 실행 불가	부록 B-2에 원인 · 수정

주의

강사 병합본에 정렬 코드가 없습니다. 13_07_combined.py는 13_07~13_12(불리언 시리즈 + 실습 1~4)까지이고, sort_values 시연과 실습 5·6 코드가 빠져 있습니다. 전사본 81~97% 구간에 정렬 수업이 온전히 담겨 있어 교안 p34~p43과 교차 복원했으므로 내용은 정확합니다.

3. 어제와의 연결 — 위치에서 조건으로

18일차에 loc·iloc으로 위치를 알 때 고르는 법을 배웠고, 마지막 15분에 조건 필터링을 살짝 열어 봤습니다. 오늘은 그 문을 끝까지 엽니다.

18일차 (어제)	19일차 (오늘)	달라지는 지점
df.loc[0:2] 위치를 알 때	df[df['두께']>=13] 성질만 알 때	위치 → 내용 기준
df['비스킷두께'] >= 13 맛보기	본격 — 개수·비율·추출까지	어제 문만 열었던 부분
16일차 넘파이 (a)&(b)	(df['A']>=1) & (df['B']>=2)	규칙이 완전히 같음
sort_values() STEP 5에서 미리 씬	정식 — 다중 키·চেই닝까지	라벨이 뒤섞이는 이유의 정체
df[['A', 'B']] 두 겹	필터 결과에도 그대로 적용	조합해서 씬
(없음)	.copy() · SettingWithCopyWarning	오늘 처음 나옴

CONTENTS

목차

오늘 배운 것 전체 지도

장 · STEP	무엇을 배우나	핵심
CHAPTER 0	오늘의 지도 — 분석 워크플로우 5단계와 용어	불러오기→확인→필터→정렬→선택
STEP 1	Boolean Series — 조건을 배열로	<code>s >= 13 .sum()</code>
STEP 2	임계값으로 골라내기 — <code>df[조건]</code>	<code>df[df['두께']>=16]</code>
STEP 3	and·or가 안 되는 이유 — <code>&</code> <code>or</code> 와 괄호	<code>(A) & (B)</code>
STEP 4	조건 도구 넷 — <code>~</code> · <code>isin</code> · <code>between</code> · <code>contains</code>	<code>~</code> · <code>isin</code> · <code>between</code>
STEP 5	조건 필터링 정리 — 바깥은 고정, 안만 바뀝니다	<code>df[어떤 조건이든]</code>
STEP 6	<code>sort_values</code> — 위험한 순서로 줄 세우기	<code>ascending=False</code>
STEP 7	다중 키 정렬과 메서드 체이닝	<code>.sort_values(...).head(5)</code>
STEP 8	<code>copy</code> 와 <code>SettingWithCopyWarning</code>	<code>df[조건].copy()</code>
STEP 9	분석 워크플로우 5단계 — 오늘의 결론	도구를 엮는 흐름
부록 A	치트시트 — 조건·정렬 도구 전체 요약	한 장 요약
부록 B	오류·함정 + 제출 코드 교정 + 교안 예상 결과 대조	$\times \rightarrow \checkmark$
부록 C	실습 1~6 하이라이트 · 실습 7 · 다음 시간 예고	그룹 집계

오늘 하루의 흐름 — 거르고, 줄 세우고

교안 p48이 이 회차의 요지를 한 문장으로 정리합니다 — "도구 하나하나보다 그것을 엮는 흐름이 분석의 진짜 힘". 오늘 배우는 도구는 둘뿐이지만, 엮으면 "가장 위험한 설비 5대를 찾아라"에 곧바로 답할 수 있습니다.



오늘의 지도

분석 워크플로우 5단계 · 오늘 나온 이름 전체

설비 적용

오늘도 **주조 다이캐스팅** 데이터를 씁니다. 어제와 같은 두 파일인데 오늘은 큰 쪽(13_diecasting_shot.csv, 200행)을 주로 씁니다. 품질등급 분포가 **양품 162 · 주의 18 · 불량 20**이라, "불량만 골라 위험도 순으로 줄 세우기" 같은 실전에 가까운 실습이 가능합니다.

0-1. 위치 기준과 조건 기준 — 무엇이 다른가

어제 배운 `loc·iloc`은 어디에 있는지 알 때 쓰는 도구였습니다. 그런데 현장의 질문은 대부분 위치를 모릅니다.

"비스킷두께가 위험하게 높은 샷이 어디야?"라는 질문에는 행 번호가 들어 있지 않습니다.

구분		
언제 쓰나	위치나 이름을 알 때	위치는 몰라도 성질을 알 때
질문 형태	"3번 행을 보자"	"비스킷두께 13 이상인 샷은?"
기준	표에서의 자리	데이터의 내용
현장 빈도	제한적 — 최신 5건 정도	압도적으로 높음
결과 크기	내가 정한 만큼	조건에 따라 달라짐

개념

강사가 어제 든 예가 명확했습니다 — "3만 개 중에 2만 5800번째 것을 굳이 뽑아낼 이유가 있나?" 실무에서 훨씬 자주 쓰는 것은 **조건에 맞는 것만 따로 뽑기**입니다. 결측이 있는 행만, 특정 설비만, 특정 날짜 구간만.

0-2. 분석 워크플로우 5단계

교안 p50~p52가 오늘 배우는 것을 포함해 **분석 전체를 다섯 단계**로 묶습니다. 한 문장으로 외우면 이렇습니다 —

"불러오고, 확인하고, 거르고, 줄 세우고, 골라냅니다."

단계	하는 일	도구	배운 회차
① 불러오기	CSV를 DataFrame으로	<code>pd.read_csv()</code>	17일차

단계	하는 일	도구	배운 회차
② 확인	구조와 자료형 점검	head · info · columns	17·18일차
③ 필터링	조건을 걸어 관심 행만	df[조건]	오늘
④ 정렬	위험 순으로 우선순위	sort_values	오늘
⑤ 선택	필요한 열 · 상위 N개만	df[['A', 'B']] · .head(5)	18일차

개념

교안 p49가 순서의 이유까지 짚습니다 — "먼저 거르고 줄 세운 뒤 마지막에 열을 추립니다." 수천 개를 다 정렬하지 않고 일부만 거른 뒤 정렬하는 편이 효율적이기 때문입니다. 순서를 바꾸면 결과는 같아도 느려집니다.

0-3. 오늘 나온 이름 전체

분류	이름	한 줄
자료형	Boolean Series	행마다 참·거짓이 담긴 시리즈
문법	df[조건]	조건이 참인 행만 추출 — 모든 필터링의 기본 패턴
메서드	.sum() · .mean()	참 개수 · 참 비율
연산자	&	그리고 — 각 조건을 괄호로 감싸야 함
연산자		또는 — 각 조건을 괄호로 감싸야 함
연산자	~	부정 — 조건 전체를 뒤집음
메서드	.isin([목록])	목록 안에 있는지 한 번에 검사
메서드	.between(소, 대)	범위 검사 — 양쪽 경계 모두 포함
메서드	.str.contains("글자")	글자 열에서 포함 여부 검사
메서드	.sort_values("열")	그 열 기준으로 줄 세우기 — 기본 오름차순
키워드	ascending=False	내림차순 — 큰 값부터
키워드	by=["열1", "열2"]	다중 키 정렬 — 적은 순서가 우선순위
개념	메서드 체이닝	점으로 동작을 이어 붙이기
메서드	.copy()	원본과 분리된 독립 복사본
개념	SettingWithCopyWarning	원본인지 복사본인지 모호할 때 뜨는 경고

STEP 1

Boolean Series

조건을 배열로 바꾸기

필터링의 첫 단계는 **값을 골라내는 것이 아니라 조건에 맞는 위치를 표시하는 것**입니다. 16일차 넘파이에서 배운 불리언 배열과 정확히 같은 원리이고, 강사도 "판다스의 새 문법이 아니라 넘파이 기법을 그대로 가져와 쓰는 것" 이라고 못 박았습니다.

1-1. 열에 조건을 걸면 참거짓 시리즈가 생깁니다

자료형 Boolean Series 행마다 참·거짓이 담긴 시리즈

정의 열 하나에 비교 연산을 걸면 **원본과 같은 길이의 참거짓 시리즈**가 만들어집니다. 어느 행이 조건에 맞는지 알려 주는 지도 역할입니다.

형태 `df['비스킷두께'] >= 13`

```
import pandas as pd

df = pd.read_csv("data/13_diecasting_small.csv")
s = df["비스킷두께"]          # 열 하나 → Series

s_boolean = s >= 13          # 조건 → Boolean Series
print(s_boolean.head())
print(s_boolean.dtype)
```

```
▶ 0    False
▶ 1    False
▶ 2     True
▶ 3    False
▶ 4     True
▶ Name: 비스킷두께, dtype: bool
▶ bool
```

왜 쓰나 조건에 맞는 값을 바로 꺼내는 것이 아니라, **먼저 위치를 표시**합니다. 이 중간 단계를 건너뛰고 이해하려 하면 `df[df["두께"]>=13]`이 왜 그렇게 생겼는지 영영 잡히지 않습니다.

응용 dtype이 bool로 바뀌는 것에 주목하십시오. **원래 값이 사라지고 위치 정보만 남은 상태**입니다. 이 불리언 시리즈 자체가 다음 단계의 재료가 됩니다.

```
s = df["비스킷두께"]
print("원본 :", s.head(5).tolist())
print("표시 :", (s >= 13).head(5).tolist())
print("크기 :", s.size, (s >= 13).size, " ← 같습니다")
```

```
▶ 원본 : [10.0, 11.0, 21.0, 11.0, 14.0]
▶ 표시 : [False, False, True, False, True]
▶ 크기 : 30 30 ← 같습니다
```

개념

비교 연산자는 파이썬 기초에서 배운 그대로입니다 — $>$ $<$ $>=$ $<=$ $==$ $!=$. 달라지는 것은 결과가 값 하나가 아니라 **시리즈**라는 점뿐입니다.

1-2. sum과 mean — 개수와 비율

메서드 `.sum()` · `.mean()` (불리언 시리즈에) 조건을 만족하는 개수와 비율

정의 True는 1, False는 0으로 계산됩니다. 합계는 개수, 평균은 전체 대비 비율이 됩니다.

```
s_boolean = df["비스킷두께"] >= 13

print("개수:", s_boolean.sum())
print("비율:", round(s_boolean.mean(), 2))
print("전체:", len(df))
```

```
▶ 개수: 6
▶ 비율: 0.2
▶ 전체: 30
```

왜 쓰나 "전체의 몇 %가 기준을 넘었나" 가 설비 보고의 표준 형식입니다. 16일차 넘파이에서 배운 것과 완전히 같은 원리입니다.

응용 개수와 비율을 한 줄로 묶어 리포트 형태로 만들어 두면 그대로 쓸 수 있습니다. 조건과 임계값만 바꾸면 어떤 열에도 적용됩니다.


```
def report(df, col, limit):
    m = df[col] >= limit
    print(f"{col} {limit} 이상: {m.sum()}건 / {len(df)}건 "
          f"({m.mean()*100:.1f}%)")
```

```
report(df, "비스킷두께", 13)
report(df, "실린더압력", 230)
report(df, "사이클타임", 25)
```

- ▶ 비스킷두께 13 이상: 6건 / 30건 (20.0%)
- ▶ 실린더압력 230 이상: 5건 / 30건 (16.7%)
- ▶ 사이클타임 25 이상: 6건 / 30건 (20.0%)

개념

`.sum()`이 "몇 개인가"를 묻는다면, 16일차에 배운 `.any()`는 "있기는 한가", `.all()`은 "전부인가"를 묻습니다. 셋 다 불리언 시리즈를 값 하나로 줄여 줍니다.

1-3. 실습 1 — 조건에서 추출까지

교안 실습 1은 **조건 개수와 추출 행 수가 같은지** 확인하는 문제입니다. 두 숫자가 같아야 필터링이 제대로 된 것입니다.

실습 1 — 참 개수와 추출 행 수가 일치

```
import pandas as pd

df = pd.read_csv("data/13_diecasting_small.csv")

# ① 비교 연산으로 Boolean Series 생성
s_boolean = df["실린더압력"] >= 230
print("dtype:", s_boolean.dtype)

# ② sum 으로 조건을 만족하는 행 개수
print("참 개수:", s_boolean.sum())

# ③ 대괄호에 넣어 행 추출
df_sub = df[df["실린더압력"] >= 230]
print("추출 행:", len(df_sub))
print(df_sub[["샷", "실린더압력", "품질등급"]])
```

```
▶ dtype: bool
▶ 참 개수: 5
▶ 추출 행: 5
▶ 샷 실린더압력 품질등급
▶ 7      8  235.0   주의
▶ 12     13  265.0   주의
▶ 17     18  265.0   주의
▶ 22     23  265.0   주의
▶ 27     28  265.0   주의
```

설비 적용

추출된 5건이 **전부 주의 등급**입니다. 실린더압력 230 이상이 곧 품질 이상 신호라는 뜻입니다. 조건 하나 걸었을 뿐인데 벌써 분석다운 결론이 나옵니다. 참고로 교안은 이 실습의 예상 결과를 "19건"이라고 적어 두었는데 실제와 다릅니다 — 부록 B-3에서 다룹니다.

STEP 2

임계값으로 골라내기

df[조건] — 모든 필터링의 기본 패턴

2-1. 체로 거르듯 추출합니다

문법 df[조건] 조건이 참인 행만 남기기

정의 앞에서 만든 참거짓 시리지를 **DataFrame 대괄호에 넣으면** 참인 행만 남습니다. 결과도 다시 DataFrame입니다.

형태 df[df['비스킷두께'] >= 16]

```
import pandas as pd

df = pd.read_csv("data/13_diecasting_small.csv")

df_sub = df[df["비스킷두께"] >= 16]
print("추출:", len(df_sub), "행")
print(df_sub[["샷", "비스킷두께", "품질등급"]])
```

▶ 추출: 5 행

▶ 샷 비스킷두께 품질등급

▶ 2	3	21.0	주의
▶ 12	13	19.0	주의
▶ 17	18	19.0	주의
▶ 22	23	19.0	주의
▶ 27	28	19.0	주의

왜 쓰나 체로 거르듯 참인 행만 통과시킵니다. 교안의 표현대로 참이면 남기고 거짓이면 버립니다.

응용 두 단계로 나눠 생각하면 헛갈리지 않습니다. ① **안쪽이 참거짓 목록을 만들고** ② **바깥 대괄호가 참인 행만 꺼냅니다.** 조건을 변수에 담아 두 줄로 쓰면 그 구조가 눈에 보입니다.

```
# 두 줄로 쪼개 쓰면 구조가 보입니다
mask = df["비스킷두께"] >= 16      # ① 조건 = Boolean Series
df_sub = df[mask]                  # ② 참인 행만 추출

print(type(mask).__name__, mask.dtype)
print(type(df_sub).__name__, df_sub.shape)

▶ Series bool
▶ DataFrame (5, 7)
```

개념

mask(마스크)는 실무에서 흔히 쓰는 변수 이름입니다. 조건 시리즈가 원본 위에 덮이는 가림막 같은 역할을 하기 때문입니다.

2-2. 거른 결과에서 필요한 열만

필터링 결과도 DataFrame이므로 **18일차에 배운 열 선택이 그대로** 적용됩니다. 열이 7개나 되면 화면이 정신없으니 볼 것만 골라냅니다.

실습 2 — 결과에서 두 열만 골라 보기

```
df_sub = df[df["비스킷두께"] >= 16]

# x 한 겹씩 따로 뽑으면 열마다 따로 출력됩니다
# print(df_sub["샷"].head(3))

# ✓ 두 겹으로 한 번에
print(df_sub[["샷", "비스킷두께"]].head(3))

▶ 샷 비스킷두께
▶ 2      3    21.0
▶ 12     13    19.0
▶ 17     18    19.0
```

설비 적용

여기서 **18일차의 대괄호 한 겹·두 겹**이 다시 나옵니다. 한 겹은 Series라 열마다 따로 출력해야 하고, 두 겹은 DataFrame이라 나란히 보입니다. 그리고 추출된 5건이 모두 **주의 등급**입니다. STEP 1의 실린더압력 조건에서 나온 5건과도 상당 부분 겹칩니다(샷 13·18·23·28). **압력이 높으면 비스킷두께도 두꺼워지는 관계**가 있다는 신호이고, 이런 두 지표의 관계를 수치로 재는 것이 다음 주에 배울 상관관계입니다.

STEP 3

and·or가 안 되는 이유

& 와 | 그리고 괄호

교안 p15가 "노베이스가 거의 100% 막히는 지점" 이라고 표시한 곳입니다. 조건이 하나면 쉬운데 두 개를 묶는 순간 오류가 납니다.

3-1. 왜 and가 안 되는가

파이썬의 and·or는 **값 하나끼리** 묶는 도구입니다. 참인지 거짓인지가 딱 정해진 값 두 개를 받습니다. 그런데 Boolean Series는 행마다 참거짓이 잔뜩 들어 있어서, and는 이것을 하나로 단정할 수 없습니다.

값 하나 vs 시리즈

```
import pandas as pd
df = pd.read_csv("data/13_diecasting_small.csv")

# 파이썬 기본 — 값 하나끼리는 잘 됩니다
a, b = 10, 5
if a > 5 and b < 3:
    print("그리고")

# x Boolean Series 에 and 를 쓰면
# df[(df["비스킷두께"] >= 13) and (df["사이클타임"] >= 25)]
# ValueError: The truth value of a Series is ambiguous.
# Use a.empty, a.bool(), a.item(), a.any() or a.all().
```

개념

오류 메시지의 "**ambiguous(모호하다)**" 가 정확히 그 뜻입니다. 30행짜리 시리즈에 참과 거짓이 섞여 있는데, 이것 전체가 참이냐 거짓이냐고 물으면 답할 수 없습니다. 16일차 넘파이에서 만난 오류와 **완전히 같은** 메시지입니다.

3-2. & 와 | — 행마다 짝을 맞춰 묶습니다

연산자 (A) & (B) · (A) | (B) 두 조건을 행 단위로 묶기

정의 &는 둘 다 참인 행만, |는 하나라도 참인 행을 남깁니다. **각 조건은 반드시 괄호로 감쌉니다.**

형태 df[(조건1) & (조건2)] · df[(조건1) | (조건2)]

```
df = pd.read_csv("data/13_diecasting_small.csv")

sub1 = df[df["비스킷두께"] >= 13]
sub2 = df[df["사이클타임"] >= 25]
print("두께만:", len(sub1), "· 타임만:", len(sub2))

both = df[(df["비스킷두께"] >= 13) & (df["사이클타임"] >= 25)]
either = df[(df["비스킷두께"] >= 13) | (df["사이클타임"] >= 25)]
print("그리고:", len(both), "· 또는:", len(either))
```

```
▶ 두께만: 6 · 타임만: 6
▶ 그리고: 5 · 또는: 7
```

왜 쓰나 실무 조건은 거의 항상 복합입니다. **정상 범위 판정**은 **&**, 여러 위험 신호 중 하나라도 걸리면 **|** 를 씁니다.

응용 개수 관계를 확인해 두면 결과를 검산할 수 있습니다. **그리고** $\leq$ 각 조건 $\leq$ **또는** **이고**, 또는 $= A + B -$ **그리고**입니다. 위 예에서 $6 + 6 - 5 = 7$ 로 정확히 맞습니다.

```
a = (df["비스킷두께"] >= 13)
b = (df["사이클타임"] >= 25)

print("A          :", a.sum())
print("B          :", b.sum())
print("A & B       :", (a & b).sum())
print("A | B       :", (a | b).sum())
print("검산 A+B-교집합:", a.sum() + b.sum() - (a & b).sum())
```

```
▶ A          : 6
▶ B          : 6
▶ A & B       : 5
▶ A | B       : 7
▶ 검산 A+B-교집합: 7
```

자주 하는 실수

괄호를 빼면 오류입니다. `df[df["두께"]>=13 & df["타임"]>=25]`처럼 쓰면 **&**가 비교보다 먼저 계산되려 해서 `TypeError: Cannot perform 'rand_' with a dtypes [float64] array and scalar of type [bool]`가 납니다.

3-3. 파이썬 기본과 Pandas의 대응

뜻	파이썬 기본	Pandas · 넘파이	주의
그리고	and	&	각 조건을 괄호로
또는	or		각 조건을 괄호로
부정	not	~	조건 전체를 괄호로

주의

16일차에 넘파이에서 배운 규칙과 **한 글자도 다르지 않습니다**. Series는 곧 넘파이 1차원 배열이기 때문입니다. 그때 손에 익혀 두었다면 오늘은 그대로 가져다 쓰면 됩니다.

3-4. 실습 3 — 두 조건 묶기

실습 3 — 200행 데이터에서

```
df = pd.read_csv("data/13_diecasting_shot.csv")    # 200행

print("두께 >= 13 :", len(df[df["비스킷두께"] >= 13]))
print("타임 >= 25 :", len(df[df["사이클타임"] >= 25]))

both = df[(df["비스킷두께"] >= 13) & (df["사이클타임"] >= 25)]
either = df[(df["비스킷두께"] >= 13) | (df["사이클타임"] >= 25)]
print("그리고(&) :", len(both), " · 또는(|):", len(either))
```

▶ 두께 >= 13 : 103
▶ 타임 >= 25 : 84
▶ 그리고(&) : 83 · 또는(|): 104
▶ 계산: 103 + 84 - 83 = 104 ✓

설비 적용

&가 83건, |가 104건입니다. 두께 조건 103건 중 83건이 타임 조건에도 걸린다는 뜻이고, 비율로는 80%입니다. 두 지표가 함께 움직인다는 강한 신호입니다. 개수 차이만 봐도 데이터의 성격이 드러납니다.

STEP 4

조건 도구 넷

~ · isin · between · str.contains

&와 |만으로도 어지간한 조건은 표현됩니다. 그런데 자주 나오는 형태에는 더 짧고 읽기 쉬운 전용 도구가 준비되어 있습니다. 네 가지를 익히면 조건 표현이 훨씬 간결해집니다.

4-1. ~ — 조건을 통째로 뒤집기

연산자 ~ (조건) 참과 거짓을 뒤집기

정의 물결 기호를 조건 앞에 붙이면 참거짓이 뒤집힙니다. 파이썬의 not에 해당하지만 **Boolean Series**에는 ~를 씁니다.

형태 df[~(df['품질등급'] == '불량')]

```
import pandas as pd
df = pd.read_csv("data/13_diecasting_shot.csv")

print("불량 :", len(df[df["품질등급"] == "불량"]))
print("불량 외:", len(df[~(df["품질등급"] == "불량")]))
print("전체 :", len(df))
```

```
▶ 불량 : 20
▶ 불량 외: 180
▶ 전체 : 200
```

왜 쓰나 예외를 제외하고 볼 때 유용합니다. 조건이 복잡할수록 이득이 큼니다 — 부등호를 하나하나 뒤집고 &를 |로 바꾸는 대신 물결 한 글자면 끝납니다.

응용 복합 조건 전체를 뒤집을 때 진가가 나옵니다. 아래 두 줄은 같은 결과인데, 위쪽은 조건을 손으로 뒤집었고 아래쪽은 ~로 통째로 뒤집었습니다.


```
a = (df["비스킷두께"] >= 13)
b = (df["사이클타임"] >= 25)

# 손으로 뒤집기 — 부등호도 바꾸고 & 도 | 로
print(len(df[(df["비스킷두께"] < 13) | (df["사이클타임"] < 25)]))

# ~ 로 통째로 뒤집기
print(len(df[~(a & b)]))

# 두 결과가 다릅니다 — 결측(NaN) 때문입니다
print("결측:", df["비스킷두께"].isna().sum(), "건")

▶ 103
▶ 117
▶ 결측: 14 건
```

주의

`not` (조건)처럼 쓰면 안 됩니다. `and`·`or`와 같은 이유로 `ValueError`가 납니다. **Series에는 ~. 키보드에서 보통 숫자 1 왼쪽에 있습니다.**

4-1-1. 결측이 섞이면 ~ 와 부등호 뒤집기가 달라집니다

위 응용 예제에서 두 결과가 **103과 117로 같았습니다**. 논리적으로는 같아야 하는데 다릅니다. 원인은 이 데이터에 있는 **결측(NaN)**입니다.

합이 전체와 맞지 않습니다

```
import pandas as pd
df = pd.read_csv("data/13_diecasting_shot.csv")

print("전체 행 :", len(df))
print("비스킷두께 >= 13:", (df["비스킷두께"] >= 13).sum())
print("비스킷두께 < 13:", (df["비스킷두께"] < 13).sum())
print("두 개의 합 :", (df["비스킷두께"] >= 13).sum() + (df["비스킷두께"] < 13).sum())
print("결측 개수 :", df["비스킷두께"].isna().sum())
```

- ▶ 전체 행 : 200
- ▶ 비스킷두께 >= 13: 103
- ▶ 비스킷두께 < 13: 83
- ▶ 두 개의 합 : 186 ← 200이 아닙니다
- ▶ 결측 개수 : 14

NaN은 어떤 비교에도 참이 되지 않습니다. NaN >= 13도 거짓이고 NaN < 13도 거짓입니다. 그래서 두 조건 어디에도 안 들어가고, 합이 전체보다 14만큼 모자랍니다.

표현	NaN 행을	결과	왜
df["두께"] < 13	제외	83건	비교 자체가 거짓
~(df["두께"] >= 13)	포함	97건	거짓을 뒤집으면 참
df["두께"].isna()	그것만	14건	결측만 골라냄

자주 하는 실수

~는 결측 행까지 참으로 만듭니다. "불량이 아닌 것"을 ~(df["품질등급"]=="불량")으로 뽑으면 품질등급이 비어 있는 행도 함께 딸려 옵니다. 이 데이터는 품질등급에 결측이 없어 문제가 안 생기지만, 숫자 열에 ~를 쓸 때는 반드시 결측을 먼저 확인해야 합니다.

설비 적용

13_diecasting_shot.csv는 200행 중 14행의 센서값 5개가 통째로 비어 있습니다(실린더압력·주조압력·사이클타임·비스킷두께·형체력). 샷 번호와 품질등급은 남아 있으니, 측정 장비나 통신이 끊겨 그 샷의 센서 기록만 유실된 상황으로 읽힙니다. 17일차에 배운 NaN의 의미가 실제 데이터에서 그대로 나타난 사례입니다. 참고로 작은 파일(13_diecasting_small.csv, 30행)에는 결측이 하나도 없습니다.

4-2. isin — 여러 값 중 하나인지

메서드 `.isin([목록])` 값이 목록 안에 있는지 한 번에 검사

정의 파이썬 `in`의 데이터판입니다. 리스트를 넘기면 **그중 하나라도 해당하는 행에** 참이 됩니다.

형태 `df['품질등급'].isin(['양품', '주의'])`

```
# 세로 막대로 잇는 방식
print(len(df[(df["품질등급"] == "양품") | (df["품질등급"] == "주의")]))

# isin 으로 — 같은 결과, 훨씬 깔끔합니다
print(len(df[df["품질등급"].isin(["양품", "주의"])]))

▶ 180
▶ 180
```

왜 쓰나 분류가 있는 데이터에서 **여러 종류 중 몇 개만 골라낼 때** 씁니다. 값이 많을수록 장점이 커집니다 — 세 종류를 고르려면 `|`를 두 번 써야 하지만 `isin`은 리스트에 넣기만 하면 됩니다.

응용 `~`와 결합하면 **목록에 없는 나머지를** 뽑습니다. 점검이 끝난 설비를 목록으로 두고 아직 안 한 것만 추릴 때 편리합니다.

```
# 목록에 있는 것
print("양품·주의 :", len(df[df["품질등급"].isin(["양품", "주의"])]))

# 목록에 없는 나머지
print("그 외 :", len(df[~df["품질등급"].isin(["양품", "주의"])]))

# 어떤 값들이 있는지 먼저 확인하는 습관
print("고유값 :", df["품질등급"].unique().tolist())

▶ 양품·주의 : 180
▶ 그 외 : 20
▶ 고유값 : ['양품', '주의', '불량']
```

자주 하는 실수

괄호 안에 리스트를 넣어야 합니다. `isin("양품")`처럼 문자열을 그냥 넣으면 글자 하나하나를 목록으로 해석해 엉뚱한 결과가 나옵니다. 대상도 반드시 **열 하나**여야 합니다 — 부록 B-2에서 다루는 실수입니다.

4-3. between — 범위 조건

메서드 `.between(소, 대)` 값이 두 경계 사이에 있는지

정의 같은 열에 하한과 상한이 있을 때 한 줄로 검사합니다. **양쪽 경계를 모두 포함합니다.**

형태 `df['실린더압력'].between(210, 230)`

```
# 복합 조건으로 쓰면
print(len(df[(df["실린더압력"] >= 210) & (df["실린더압력"] <= 230)]))

# between 으로 — 같은 결과, 열 이름을 한 번만
print(len(df[df["실린더압력"].between(210, 230)]))

# 범위 밖
print(len(df[~df["실린더압력"].between(210, 230)]))
```

- ▶ 89
- ▶ 89
- ▶ 111

왜 쓰나 같은 열이 두 번 반복되지 않아 **짧고 읽기 쉽습니다.** 정상 범위를 정의할 때 자연스러운 표현입니다.

응용 `~between`은 **정상 범위 밖의 이상치**를 골라내는 표현이 됩니다. 16일차에 배운 3-시그마와 결합하면 임계값을 데이터가 스스로 정하게 할 수도 있습니다.

```
s = df["실린더압력"]
lo, hi = round(s.mean() - 2*s.std(), 1), round(s.mean() + 2*s.std(), 1)

print(f"2시그마 정상 범위: {lo} ~ {hi}")
print("범위 안   :", s.between(lo, hi).sum(), "건")
print("범위 밖   :", len(df[~s.between(lo, hi)]), "건")
print("  그중 결측 :", s.isna().sum(), "건")
```

- ▶ 2시그마 정상 범위: 169.9 ~ 298.9
- ▶ 범위 안 : 180 건
- ▶ 범위 밖 : 20 건
- ▶ 그중 결측 : 14 건

주의

양 끝을 포함한다는 점이 중요합니다. `between(210, 230)`을 "210 초과 230 미만"으로 착각하면 경계값이 빠져 결과가 미묘하게 달라집니다. 이 데이터에는 정확히 210이나 230인 값이 없어 차이가 안 보이지만, 경계값이 많은 데이터에서는 크게 갈립니다.

4-4. str.contains — 글자 열에서 포함 검사

메서드 `.str.contains("글자")` 글자 열에서 특정 단어 포함 여부

정의 `==`가 정확히 일치하는 반면, `contains`는 **일부라도 포함**하면 참이 됩니다.

형태 `df['품질등급'].str.contains('양')`

```
# == 은 정확히 일치해야 합니다
print("== '양' : ", len(df[df["품질등급"] == "양"]))

# contains 는 포함만 하면 됩니다
print("contains('양'):", len(df[df["품질등급"].str.contains("양")]))
```

```
▶ == '양' : 0
▶ contains('양'): 162
```

왜 쓰나 설비 ID나 로트 번호처럼 **패턴이 들어 있는 글자 열**에서 특정 계열만 골라낼 때 씁니다. MCT-01, MCT-02 중 MCT 계열만 뽑는 식입니다.

응용 `.str`은 **이 열을 글자로 다루겠다**는 신호입니다. 숫자 열에 쓰면 오류가 나므로, `dtypes`로 `object`인지 먼저 확인하는 습관이 좋습니다.

```
print(df["품질등급"].dtype, "<- object 여야 .str 사용 가능")

# 숫자 열에 쓰면 오류
# df["실린더압력"].str.contains("2")
# AttributeError: Can only use .str accessor with string values
```

```
▶ object <- object 여야 .str 사용 가능
```

개념

글자 열에 결측(NaN)이 있으면 `contains`가 오류를 냅니다. 그럴 때는 `na=False`를 주어 결측을 거짓으로 처리합니다 — `df["열"].str.contains("A", na=False)`.

4-5. 조건 도구 한눈에

도구	용도	예시	대체 표현
&	모두 만족	(A) & (B)	없음 — 필수
	하나라도 만족	(A) (B)	isin으로 대체 가능한 경우 있음
~	부정·뒤집기	~(조건)	부등호를 손으로 뒤집기
.isin()	목록 매칭	.isin(['양품', '주의'])	를 여러 번
.between()	범위 검사	.between(210, 230)	(>=) & (<=)
.str.contains())	글자 포함	.str.contains('양')	없음

STEP 5

조건 필터링 정리

바깥은 고정, 안만 바뀝니다

교안 p30이 이 파트를 한 줄로 요약합니다 — "모든 조건 필터링의 기본 패턴은 `df[조건]`". 단일 조건이든 복합 조건이든, 결국 이 패턴 안에 어떤 조건을 넣느냐의 문제입니다.

일곱 가지가 전부 같은 모양입니다

```
df[ df["비스킷두께"] >= 13 ] # 단일
df[ (df["두께"] >= 13) & (df["타임"] >= 25) ] # 그리고
df[ (df["두께"] >= 13) | (df["타임"] >= 25) ] # 또는
df[ ~(df["품질등급"] == "불량") ] # 부정
df[ df["품질등급"].isin(["양품", "주의"]) ] # 목록
df[ df["실린더압력"].between(210, 230) ] # 범위
df[ df["품질등급"].str.contains("양") ] # 포함
# ↑ 바깥 구조는 언제나 df[ ] — 안의 조건만 바뀝니다
```

5-1. 두 단계로 정리

단계	무엇을	도구
① 조건 만들기	비교로 Boolean Series 생성	<code>></code> · <code><</code> · <code>==</code> · <code>isin</code> · <code>between</code> · <code>contains</code>
② 조건 묶기	여러 조건을 결합·뒤집기	<code>&</code> · <code> </code> · <code>~</code>
③ 추출	참인 행만 꺼내기	<code>df[조건]</code>

5-2. 가장 흔한 두 가지 실수

실수	증상	바로잡기
① and·or	ValueError: truth value ... ambiguous	<code>&</code> · <code> </code> 로 바꾸기
② 괄호 누락	TypeError: Cannot perform 'rand_' ...	(조건1) & (조건2) 형태를 몸에 익히기

STEP 6

sort_values

위험한 순서로 줄 세우기

필터링이 **이상 설비를 찾는 일**이라면, 정렬은 **무엇부터 손댈지 정하는 일**입니다. 교안 p34가 둘을 "한 짝"이라고 부르는 이유입니다. 참고로 교안 p32는 앞 파트를 이렇게 답합니다 — "**이 조건 필터링 기술은 결측치·이상치·고장 예측까지 모든 분석의 기초**".

6-1. 정렬이 필요한 이유

이유	설명	현장에서
집중의 한계	사람이 한 번에 볼 수 있는 대상은 제한적	100대를 동시에 점검할 수 없으니 시급한 것부터
흐름 파악	정렬하면 숨은 패턴이 드러남	시간순은 변화 흐름, 값순은 극단값

메서드 `.sort_values("열")` 그 열 기준으로 줄 세우기

정의 괄호에 기준 열 이름을 적으면 그 열로 정렬합니다. 기본은 작은 값부터인 오름차순입니다.

형태 `df.sort_values('비스킷두께')` · `df.sort_values('비스킷두께', ascending=False)`

```
import pandas as pd
df = pd.read_csv("data/13_diecasting_small.csv")

# 큰 값부터 — 위험 우선 보기
top = df.sort_values("비스킷두께", ascending=False)
print(top.head(3)[["샷", "비스킷두께", "품질등급"]])
```

```
▶ 샷 비스킷두께 품질등급
▶ 2      3    21.0  주의
▶ 12     13    19.0  주의
▶ 22     23    19.0  주의
```

왜 쓰나 비스킷두께 큰 순으로 줄 세우면 가장 위험한 설비가 맨 위로 올라옵니다. 어디부터 손대야 할지가 한눈에 정해집니다.

응용 두 가지 성질을 반드시 알아 두어야 합니다. ① **한 열로 정렬해도 모든 열이 행 단위로 함께 이동합니다.** 엑셀에서 한 열만 정렬하다 데이터가 어긋나는 사고가 없습니다. ② **원본은 그대로입니다.** 정렬 결과는 새로 반환되므로 마음 놓고 여러 기준으로 실험할 수 있습니다.


```
# ① 행 통째 이동 — 품질등급도 함께 따라옵니다
print(df.sort_values("비스킷두께", ascending=False).head(2))
```

```
# ② 원본 보존
print("정렬 후 원본 앞 3개:", df["비스킷두께"].head(3).tolist())
```

```
▶ 샷 실린더압력  주조압력  사이클타임  비스킷두께  형체력  품질등급
▶ 2      3  215.0  1040.0   20.7   21.0  253.0   주의
▶ 12     13  265.0   596.0   33.9   19.0  354.0   주의
▶ 정렬 후 원본 앞 3개: [10.0, 11.0, 21.0]
```

개념

왼쪽 인덱스가 2, 12, 17로 뒤섞인 것에 주목하십시오. **18일차에 배운 "정렬하면 라벨과 위치가 갈라진다"**가 바로 이 장면입니다. 이 상태에서 `loc[0:2]`를 쓰면 엉뚱한 행이 나옵니다.

6-2. 오름차순과 내림차순

방향	코드	언제 쓰나
오름차순 (기본)	<code>df.sort_values("열")</code>	작은 값이 문제일 때 — 형체력 부족, 압력 미달
내림차순	<code>df.sort_values("열", ascending=False)</code>	큰 값이 문제일 때 — 두께 과다, 사이클 지연

주의

`ascending`은 "오름차순인가?"를 묻는 질문입니다. 그래서 내림차순을 원하면 `ascending=False`입니다. `descending=True` 같은 인자는 없습니다. 헛갈리면 "ascend = 올라간다"를 떠올리십시오.

연결

결측이 있으면 **정렬 결과의 맨 뒤로 밀립니다**. 기본값이 `na_position="last"`이기 때문입니다.

13_diecasting_shot.csv처럼 결측이 14건 있는 데이터를 내림차순 정렬하면, 큰 값들이 위로 오고 결측 14행이 맨 아래에 모입니다. 앞에서 `head(5)`만 볼 때는 문제가 없지만 `tail()`을 보면 전부 NaN입니다.

다중 키 정렬과 메서드 체이닝

우선순위와 점으로 잇기

7-1. 여러 열로 정렬 — 적은 순서가 우선순위

키워드 by=["열1", "열2"] 우선순위가 있는 다중 정렬

정의 기준 열을 리스트로 묶으면 **적은 순서가 곧 우선순위**가 됩니다. 첫 열로 큰 덩어리를 나누고, 값이 같은 것끼리만 둘째 열로 세부 정렬합니다.

형태 df.sort_values(['품질등급', '비스킷두께'], ascending=[True, False])

```
import pandas as pd
df = pd.read_csv("data/13_diecasting_small.csv")

r = df.sort_values(["품질등급", "비스킷두께"], ascending=[True, False])
print(r.head(5)[["샷", "품질등급", "비스킷두께"]])
```

```
▶ 샷 품질등급 비스킷두께
▶ 4      5   불량 14.0
▶ 9     10   불량 12.0
▶ 19    20   불량 11.0
▶ 14    15   불량  4.0
▶ 24    25   불량  2.0
```

왜 쓰나 "불량부터 보되, 불량 안에서는 두꺼운 순으로" 같은 실무 요구가 그대로 표현됩니다. 우선순위가 두 단계인 보고서를 만들 때 필수입니다.

응용 ascending도 **얼마나 따로** 줄 수 있습니다. 리스트로 넘기면 첫 열은 오름차순, 둘째 열은 내림차순처럼 방향을 자유롭게 섞을 수 있습니다. 위 예에서 품질등급은 가나다순(불량→양품→주의) 오름차순, 비스킷두께는 내림차순입니다.

```
# 방향을 하나만 주면 모든 열에 같은 방향이 적용됩니다
a = df.sort_values(["품질등급", "비스킷두께"], ascending=False)
print("둘 다 내림:", a.head(1)[["품질등급", "비스킷두께"]].values.tolist())

# 열마다 다르게
b = df.sort_values(["품질등급", "비스킷두께"], ascending=[True, False])
print("등급↑ 두께↓:", b.head(1)[["품질등급", "비스킷두께"]].values.tolist())
```

▶ 둘 다 내림: [['주의', 21.0]]
 ▶ 등급↑ 두께↓: [['불량', 14.0]]

개념

글자 열의 오름차순은 **가나다순**입니다. 그래서 불량 → 양품 → 주의 순으로 나옵니다. 위험한 순서(양품→주의→불량)로 정렬하고 싶다면 등급에 숫자를 매기거나 Categorical 자료형을 써야 하는데, 그것은 이후 학습 범위입니다.

7-2. 메서드 체이닝 — 점으로 잇기

개념 메서드 체이닝 (method chaining) 동작을 점으로 이어 한 줄로

정의 앞 동작의 결과에 바로 다음 동작을 점으로 이어 붙이는 방식입니다. 왼쪽에서 오른쪽으로 읽으면 흐름이 자연스럽게 이어집니다.

형태 `df.sort_values('비스킷두께', ascending=False).head(5)`

```
df = pd.read_csv("data/13_diecasting_small.csv")

# 두 줄로 나눠 쓰면
sorted_df = df.sort_values("비스킷두께", ascending=False)
top5 = sorted_df.head(5)

# 점으로 이으면 한 줄
top5 = df.sort_values("비스킷두께", ascending=False).head(5)
print(top5[["샷", "비스킷두께"]])
```

▶ 샷 비스킷두께
 ▶ 2 3 21.0
 ▶ 12 13 19.0
 ▶ 22 23 19.0
 ▶ 27 28 19.0
 ▶ 17 18 19.0

왜 쓰나 "점검 시급 상위 10대" 처럼 중요한 일부만 추출하는 작업이 한 줄로 끝납니다. 중간 결과에 이름을 붙일 필요가 없어집니다.

응용 조건 필터링까지 이으면 실습 6이 됩니다. **거르고 → 줄 세우고 → 상위 뽑고 → 열 고르기**가 한 줄에 담깁니다. 길어지면 괄호로 감싸 여러 줄로 나눠 쓸 수 있습니다.

```
df = pd.read_csv("data/13_diecasting_shot.csv")

# 한 줄로
r = df[df["품질등급"] == "불량"].sort_values("비스킷두께", ascending=False).head(5)

# 길면 괄호로 감싸 줄바꿈 — 읽기 훨씬 좋습니다
r = (df[df["품질등급"] == "불량"]
      .sort_values("비스킷두께", ascending=False)
      .head(5))
print(r[["샷", "비스킷두께", "실린더압력"]])
```

```
▶ 샷 비스킷두께 실린더압력
▶ 197 198 27.0 113.0
▶ 191 192 26.0 113.0
▶ 196 197 20.0 265.0
▶ 193 194 19.0 113.0
▶ 198 199 19.0 264.0
```

주의

강사가 이 대목에서 **변수를 따로 뺄지 한 줄로 이을지**를 이야기했습니다. 정답은 없고 코딩 스타일 문제인데, 강사 본인은 **"명확한 이름을 붙여 따로 빼는 쪽"**을 선호한다고 했습니다. 중간 결과에 이름이 있으면 나중에 읽기 쉽기 때문입니다.

주의

위 결과에서 **비스킷두께 19.0인 네 행(샷 13·18·23·28)의 순서가 뒤죽박죽**입니다. `sort_values`의 기본 정렬 방식이 동점끼리의 순서를 보장하지 않기 때문입니다. 원본 순서를 유지하려면 `kind="stable"`을 줍니다 — `df.sort_values("비스킷두께", ascending=False, kind="stable")`. 리포트를 매번 같은 순서로 뽑아야 한다면 반드시 지정하십시오.

동점이 있을 때

동점 순서 비교

```
a = df.sort_values("비스킷두께", ascending=False)
```

```
b = df.sort_values("비스킷두께", ascending=False, kind="stable")
```

```
print("기본 :", a.head(5)["샷"].tolist())
```

```
print("stable:", b.head(5)["샷"].tolist())
```

▶ 기본 : [3, 13, 23, 28, 18]

▶ stable: [3, 13, 18, 23, 28] ← 원본 순서 유지

7-3. 좋은 코드에 대한 강사의 조언

정렬 실습 중에 나온 이야기가 오늘 수업에서 기술 외적으로 가장 값진 부분이었습니다. 그대로 정리해 둡니다.

조언	내용	왜
80줄 규칙	한 줄이 80글자를 넘지 않게	옛 프린터 한 줄이 80글자였던 관례가 지금도 가이드로 남음
이름 짓기	변수명·함수명에 신경 쓰기	강사 표현: "개발자 스트레스의 절반 이상은 이름 짓기"
주석 달기	주석을 잘 달거나, 이름을 잘 짓거나	둘 중 하나는 해야 나중에 읽힘
구조 정하기	이름만으로 한계가 있으면 구조로	함수·모듈로 나누어 흐름을 드러냄

개념

강사가 내린 좋은 코드의 정의가 명확합니다 — "잘 작동하는 것은 일차적인 기준일 뿐이고, 진짜 좋은 코드는 덮어 놓고 다음 달에 다시 열어 봤을 때 한 번에 이해되는 코드" 입니다. 중간 부분만 봤을 때도 앞에서부터 다 읽지 않고 파악되면 좋은 코드입니다.

설비 적용

포트폴리오 관점에서도 그대로 적용됩니다. 면접관이 GitHub 레포를 열었을 때 `df_1`, `df_2`, `mask_1` 같은 이름보다 `defective`, `high_thickness` 같은 이름이 훨씬 좋게 읽힙니다. 오늘 실습 코드를 정리해 커밋할 때 이름부터 손보면 완성도가 눈에 띄게 올라갑니다.

STEP 8

copy와 경고

걸러낸 결과를 수정할 때

필터링 결과를 **보기만 할 때**는 아무 문제가 없습니다. 그런데 거기에 **새 열을 추가하거나 값을 바꾸려 하면** 빨간 경고가 길게 뜹니다. 오류가 아니라 경고라서 코드는 끝까지 돌지만, 결과를 장담할 수 없다는 뜻입니다.

8-1. SettingWithCopyWarning

경고가 뜨는 상황

```
import pandas as pd
df = pd.read_csv("data/13_diecasting_shot.csv")

# 불량만 골라 위험등급 열을 붙이려 하면
sub = df[df["품질등급"] == "불량"]
sub["위험등급"] = "HIGH"

# SettingWithCopyWarning:
# A value is trying to be set on a copy of a slice from a DataFrame.
# Try using .loc[row_indexer, col_indexer] = value instead
```

이름은 길지만 뜻은 단순합니다 — "**지금 수정하려는 것이 원본의 일부인지 새 복사본인지 판단할 수 없다**" 는 것입니다. 원본이면 값을 바꿀 때 원본까지 바뀌고, 복사본이면 복사본만 바뀝니다. 어느 쪽인지 예측할 수 없다는 점이 문제입니다.

메서드 `.copy()` 원본과 분리된 독립 복사본 만들기

정의 `df[조건].copy()` 는 원본과 완전히 분리된 새 DataFrame을 만듭니다. 이후 수정은 경고 없이 수행됩니다.

형태 `sub = df[조건].copy()`

```
df = pd.read_csv("data/13_diecasting_shot.csv")

# .copy() 를 붙이면 경고가 사라집니다
sub = df[df["품질등급"] == "불량"].copy()
sub["위험등급"] = "HIGH"

print(sub[["샷", "비스킷두께", "위험등급"]].head(2))
print("원본 열 개수:", df.shape[1], "· 복사본:", sub.shape[1])
```

```
▶ 샷 비스킷두께 위험등급
▶ 180 181 14.0 HIGH
▶ 181 182 12.0 HIGH
▶ 원본 열 개수: 7 · 복사본: 8
```

왜 쓰나 위험 설비만 골라낸 뒤 등급 열을 추가하는 식의 작업이 실무에 매우 흔합니다. 원본은 그대로 두고 분석용 사본에서만 작업하는 것이 안전합니다.

응용 교안이 제시하는 판단 기준은 간단합니다 — **수정할 거면 .copy(), 보기만 하면 불필요합니다. 그리고 원본 자체를 수정할 때는 .loc 을 씁니다. 어느 행의 어느 열을 바꿀지 명확히 지정하면 판다스가 헛갈리지 않습니다.**

```
# ① 따로 작업 — .copy()
sub = df[df["품질등급"] == "불량"].copy()
sub["위험등급"] = "HIGH"      # 원본은 그대로

# ② 원본 수정 — .loc 으로 위치를 명확히
df.loc[df["품질등급"] == "불량", "위험등급"] = "HIGH"
print("원본 열 개수:", df.shape[1])
print(df["위험등급"].value_counts(dropna=False).to_dict())
```

```
▶ 원본 열 개수: 8
▶ {nan: 180, 'HIGH': 20}
```

개념

복사본은 메모리를 더 씁니다. 수백만 행짜리 데이터에서 습관적으로 .copy()를 남발하면 메모리가 부족해질 수 있으니, **수정할 때만** 붙이는 것이 원칙입니다.

8-2. 두 가지 해결법 비교

상황	방법	코드	원본은
따로 분석하겠다	<code>.copy()</code>	<code>sub = df[조건].copy()</code>	그대로 보존
원본을 고치겠다	<code>.loc</code>	<code>df.loc[조건, "열"] = 값</code>	수정됨
보기만 하겠다	그냥 두기	<code>df[조건]</code>	그대로 — 경고도 없음

주의

경고는 오류가 아닙니다. 빨간 글씨로 길게 나와 당황하기 쉽지만 코드는 멈추지 않고 끝까지 실행됩니다. 다만 결과가 의도와 다를 수 있으니 **경고를 보면 반드시 `.copy()`나 `.loc`으로 의도를 분명히 하십시오.** 무시하고 넘어가면 나중에 "왜 원본이 바뀌었지?" 하고 몇 시간을 잡아먹습니다.

STEP 9

분석 워크플로우 5단계

도구를 엮는 흐름

교안 p48이 이 회차의 결론을 말합니다 — "도구 하나하나보다 그것을 엮는 흐름이 분석의 진짜 힘". 복잡한 질문도 작은 단계로 쪼개면 오늘 배운 도구 조합으로 해결됩니다.



9-1. 한 질문을 다섯 단계로 쪼개기

교안이 든 예시 질문은 "가장 위험한 설비 5대의 샷 번호와 비스킷두께"입니다. 이것을 단계로 쪼개면 필터 → 정렬 → 상위 5 → 열 선택입니다.

다섯 단계를 하나의 흐름으로

```
import pandas as pd
```

```
# ① 불러오기
```

```
df = pd.read_csv("data/13_diecasting_shot.csv")
```

```
# ② 확인
```

```
print(f"[구조] {df.shape} · 결측 {df.isna().sum().sum()}건")
```

```
print(f"[등급] {df['품질등급'].value_counts().to_dict()}")
```

```
# ③ 필터링 — 불량만
```

```
defective = df[df["품질등급"] == "불량"]
```

```
print(f"[필터] 불량 {len(defective)}건")
```

```
# ④ 정렬 — 비스킷두께 큰 순
```

```
ranked = defective.sort_values("비스킷두께", ascending=False, kind="stable")
```

```
# ⑤ 선택 — 상위 5개, 두 열만
```

```
report = ranked.head(5)[["샷", "비스킷두께"]]
```

```
print(report.to_string(index=False))
```

▶ [구조] (200, 7) · 결측 70건

- ▶ [등급] {'양품': 162, '불량': 20, '주의': 18}
- ▶ [필터] 불량 20건
- ▶ 샷 비스킷두께
 - ▶ 198 27.0
 - ▶ 192 26.0
 - ▶ 197 20.0
 - ▶ 193 19.0
 - ▶ 194 19.0

설비 적용

결과를 읽어 보면 불량 상위 5건이 전부 샷 190번대입니다. 200번 가까이에서 문제가 몰려 있다는 뜻이고, 시간이 지나면서 상태가 나빠졌다는 신호로 읽힙니다. 조건과 정렬 두 줄로 이런 패턴이 드러납니다. 다음 주에 배울 그룹 집계를 쓰면 구간별로 나눠 더 정밀하게 확인할 수 있습니다.

9-2. 각 단계의 도구 정리

단계	도구	배운 회차	한 줄
① 불러오기	<code>pd.read_csv()</code>	17일차	<code>sep · encoding · usecols</code>
② 확인	<code>head · info · describe</code>	17·18일차	구조·결측·통계
③ 필터링	<code>df[조건]</code>	오늘	<code>& · · ~ · isin · between</code>
④ 정렬	<code>sort_values</code>	오늘	<code>ascending · 다중 키</code>
⑤ 선택	<code>df[['A', 'B']] · .head(n)</code>	18일차	두 겹 대괄호

개념

교안 p51의 마지막 문장이 좋습니다 — "각 단계마다 쓰는 도구가 정해져 있어 단계만 떠올리면 코드도 자연스럽게 떠오릅니다." 새 데이터 앞에서 막막할 때 "지금 몇 번째 단계인가"를 물으면 다음에 칠 코드가 나옵니다.

연결

교안 p52가 못 박습니다 — "이 워크플로우는 결측치·이상치·고장 예측 모델까지 모든 분석의 골격"입니다. 다음 주부터 배울 내용이 전부 이 다섯 단계 위에 얹힙니다. 오늘 것이 흔들리면 그 위가 다 흔들립니다.

치트시트

오늘 나온 전부 — 한 장 요약

A-1. 조건 만들기

표현	뜻	예	결과
<code>> · < · >= · <=</code>	크기 비교	<code>df['두께'] >= 13</code>	Boolean Series
<code>== · !=</code>	같다 · 다르다	<code>df['등급'] == '불량'</code>	Boolean Series
<code>.isin([목록])</code>	목록 매칭	<code>.isin(['양품', '주의'])</code>	Boolean Series
<code>.between(소, 대)</code>	범위 — 양 끝 포함	<code>.between(210, 230)</code>	Boolean Series
<code>.str.contains("글자")</code>	글자 포함 — object 열만	<code>.str.contains('양')</code>	Boolean Series
<code>.isna() · .notna()</code>	결측 여부	<code>df['두께'].isna()</code>	Boolean Series

A-2. 조건 묶기와 추출

표현	뜻	주의
<code>(A) & (B)</code>	둘 다 만족	각 조건에 괄호 필수 · and 안 됨
<code>(A) (B)</code>	하나라도 만족	각 조건에 괄호 필수 · or 안 됨
<code>~(A)</code>	뒤집기	결측 행까지 참이 됩니다 · not 안 됨
<code>df[조건]</code>	참인 행 추출	모든 필터링의 기본 패턴
<code>조건.sum()</code>	참 개수	불리언은 True=1
<code>조건.mean()</code>	참 비율	<code>round(_, 2)</code> 로 자르기

A-3. 정렬

표현	뜻	비고
<code>.sort_values("열")</code>	오름차순 (작은 값부터)	기본값

표현	뜻	비고
<code>.sort_values("열", ascending=False)</code>	내림차순 (큰 값부터)	descending 인자는 없습니다
<code>.sort_values(["A", "B"])</code>	다중 키 — 적은 순서가 우선순위	A로 나누고 A가 같으면 B로
<code>ascending=[True, False]</code>	열마다 방향 지정	리스트 길이를 열 수와 맞출 것
<code>kind="stable"</code>	동점 순서 유지	리포트 재현성이 필요할 때
<code>na_position="first"</code>	결측을 맨 앞으로	기본은 맨 뒤(last)

A-4. 수정할 때

상황	코드	원본은
따로 분석	<code>sub = df[조건].copy()</code>	보존
원본 수정	<code>df.loc[조건, "열"] = 값</code>	수정됨
보기만	<code>df[조건]</code>	보존 · 경고 없음

A-5. 그대로 쓰는 리포트 템플릿

복붙해서 쓰는 리포트 함수

```
import pandas as pd

def risk_report(df, cond, sort_col, n=5, cols=None):
    # 필터 → 정렬 → 상위 n개 → 열 선택
    r = df[cond].sort_values(sort_col, ascending=False, kind="stable").head(n)
    if cols:
        r = r[cols]
    print(f"조건 만족 {cond.sum()}건 중 상위 {n}건")
    print(r.to_string(index=False))
    return r

df = pd.read_csv("data/13_diecasting_shot.csv")

risk_report(df, df["품질등급"] == "불량", "비스킷두께",
            cols=["샷", "비스킷두께", "실린더압력"])
```

▶ 조건 만족 20건 중 상위 5건

▶ 샷 비스킷두께 실린더압력

```
▶ 198    27.0   113.0
▶ 192    26.0   113.0
▶ 197    20.0   265.0
▶ 193    19.0   264.0
▶ 194    19.0   113.0
```

오류·함정 사전

x 이렇게 하면 → 무슨 일이 → ✓ 바로잡기

B-1. 오늘 만나기 쉬운 오류 10가지

x 이렇게 하면	무슨 일이 일어나	✓ 바로잡기
<code>df[(A) and (B)]</code>	<code>ValueError: truth value ... ambiguous</code>	<code>df[(A) & (B)]</code>
<code>df[A & B]</code> (괄호 없이)	<code>TypeError: Cannot perform 'rand_' ...</code>	<code>df[(A) & (B)]</code> 각 조건에 괄호
<code>df[not (조건)]</code>	<code>ValueError: 같은 오류</code>	<code>df[~(조건)]</code>
<code>df[df.isin([목록])]</code>	전체 행이 남고 값이 NaN 이 됨	<code>df[df['열'].isin([목록])]</code>
<code>.isin('양품')</code> (리스트 없이)	글자 하나하나를 목록으로 해석	<code>.isin(['양품'])</code>
<code>df['숫자열'].str.contains(...)</code>	<code>AttributeError: Can only use .str ...</code>	dtypes로 object 인지 먼저 확인
<code>~(df["두께"] >= 13)</code> 결측 있을 때	결측 행까지 참이 됨 (97 vs 83)	<code>df["두께"] < 13</code> 또는 결측 먼저 처리
<code>descending=True</code>	<code>TypeError: unexpected keyword</code>	<code>ascending=False</code>
<code>sub = df[조건]</code> 후 값 수정	<code>SettingWithCopyWarning</code>	<code>sub = df[조건].copy()</code>
정렬 후 <code>loc[0:4]</code>	라벨이 뒤섞여 엉뚱한 행	<code>.head(5)</code> 또는 <code>iloc[0:5]</code>

B-2. 제출 코드 교정 — 실제로 실행해 확인한 것

① 실행이 되지 않습니다 — `SyntaxError`

`Practice/13_02_example.py`는 파일 자체가 실행되지 않습니다. 89번째 줄에서 파싱이 중단됩니다.

Practice/13_02_example.py

```
# x 제출 코드 (89행)
print(f"불량 아닌 것:\n\n\
      {df_4[ ~(df_4["품질등급"] == "불량")].head()})")
#
      ↑ f-string 바깥과 같은 큰따옴표

# SyntaxError: f-string: unmatched '['

# ✓ 안쪽을 작은따옴표로
print(f"불량 아닌 것:\n\n{df_4[~(df_4['품질등급'] == '불량')].head()})")
```

개념

f-string 중괄호 안에서 **바깥 문자열과 같은 종류의 따옴표**를 쓰면 파이썬 3.11 이하에서 오류가 납니다. 3.12부터는 허용되지만, 강의 환경과 채점 환경이 그 아래일 수 있으니 **안쪽은 항상 다른 따옴표**를 쓰는 편이 안전합니다. 더 확실한 방법은 f-string 밖에서 미리 변수에 담는 것입니다.

권장

```
# 가장 안전한 형태 — 미리 담고 출력
not_defective = df_4[~(df_4["품질등급"] == "불량")]
print("불량 아닌 것:")
print(not_defective.head())
```

② isin이 잘못된 대상에 걸려 있습니다

Practice/13_02_example.py 실습 4

```
# x 제출 코드 — DataFrame 전체에 isin
print(df_4[df_4.isin(["양품", "주의"])])
print(len(df_4[df_4.isin(["양품", "주의"])]))

# ✓ 열 하나에 isin
print(len(df_4[df_4["품질등급"].isin(["양품", "주의"])]))
```

- ▶ 샷 실린더압력 주조압력 사이클타임 비스킷두께 형체력 품질등급
- ▶ 0 NaN NaN NaN NaN NaN NaN 양품
- ▶ 1 NaN NaN NaN NaN NaN NaN 양품
- ▶ 200 ← 전체 행이 그대로 남습니다
- ▶
- ▶ 180 ← 올바른 결과

자주 하는 실수

`df.isin(...)`는 모든 칸을 검사해 참거짓 표를 만듭니다. 그것을 `df[...]`에 넣으면 행 필터가 아니라 **칸 단위 마스킹**이 되어, 조건에 안 맞는 칸이 전부 NaN으로 바뀐 채 200행이 그대로 남습니다. 오류가 안 나서 더 위험합니다. `isin`은 반드시 **열 하나**에 겁니다.

③ 잘 한 부분

부분	무엇이 좋은가
실습 2의 <code>df.loc[mask, ["샷", "비스킷두께"]]</code>	필터와 열 선택을 loc 한 줄로 — 교안보다 간결합니다
실습 3의 <code>describe()</code> 사전 확인	임계값을 정하기 전에 분포를 먼저 본 것 — 좋은 습관입니다
실습 5의 다중 정렬	<code>by=[...], ascending=[False, False]</code> 형태를 정확히 씀
실습 6의 컬럼 탐색	"고장" 값이 있는 열을 for로 찾고 없으면 대안을 안내 — 방어적으로 잘 짰습니다

설비 적용

특히 실습 6이 인상적입니다. 교안은 "고장 여부 조건"이라고만 했는데 데이터에 그런 열이 없자, **열을 훑어 "고장" 값이 있는지 찾고 없으면 품질등급의 "불량"으로 대체**한다고 코드에 명시했습니다. 실무에서 스펙과 데이터가 안 맞을 때 하는 일이 정확히 이것입니다.

B-3. 교안 예상 결과가 실데이터와 맞지 않습니다

교안 각 실습의 「예상 결과」 숫자가 제공된 두 데이터 파일 어느 쪽과도 맞지 않습니다. 강사님도 병합본 주석에 표시를 남겨 두었습니다. 직접 계산해 대조한 표입니다.

실습	조건			
실습 1	실린더압력 >= 230	19건	5	91
실습 2	비스킷두께 >= 16	40건	5	74
실습 3	(두께>=13) & (타임>=25)	12건	5	83
실습 3	(두께>=13) (타임>=25)	94건	7	104
실습 4	~(품질등급 == "불량")	192건	24	180
실습 4	<code>isin(["양품", "주의"])</code>	94건	24	180

실습	조건			
실습 4	실린더압력.between(210,230)	108건	24	89

주의

교안이 또 다른 데이터셋을 기준으로 쓰인 것으로 보입니다. 실습 4의 "192건"은 200행 기준일 때 불량이 8건이어야 나오는 숫자인데 실제 파일은 20건입니다. **이럴 때는 실행 결과가 정답입니다.** 슬라이드 숫자와 다르다고 자기 코드를 의심하지 말고, `len()` 과 `.sum()` 으로 직접 확인하십시오.

개념

강사 배포 코드에도 # 예상 결과 ... x 처럼 **x 표시**가 붙어 있습니다. 강사님도 실행해 보고 안 맞는 것을 확인한 것입니다. 원자료끼리 어긋날 때 **무엇을 기준으로 삼을지 판단하는 것도** 데이터 분석가의 일입니다.

실습 하이라이트

제출한 것 · 실습 7 · 다음 시간

C-1. 오늘 나온 실습 전체

실습	교안 요구	제출	한 줄 평
실습 1	단일 조건으로 행 추출	제출	Boolean Series → sum → 추출 순서 정확
실습 2	임계값 넘는 설비 골라내기	제출	loc 으로 필터·열선택 한 줄 처리 — 좋음
실습 3	두 조건 묶기	제출	describe 로 임계값 먼저 확인한 것이 좋음
실습 4	부정·목록·범위 조건	제출	isin 을 열이 아니라 DF 전체에 적용
실습 5	위험 순으로 정렬	제출	다중 정렬 by·ascending 리스트 정확
실습 6	필터링과 정렬 연결	제출	컬럼 탐색 로직까지 방어적으로 작성 — 좋음
실습 7	이상 의심 설비 리포트	과제	아래 보충답안

주의

제출 전에 반드시 파일을 한 번 실행해 보십시오. 현재 13_02_example.py는 SyntaxError로 한 줄도 실행되지 않습니다(부록 B-2). 실습 1~6을 다 채웠는데 채점 환경에서 아무것도 안 돌아가면 억울합니다. python Practice/13_02_example.py 한 번이면 확인됩니다.

C-2. 실습 7 보충답안 — 이상 의심 설비 리포트

교안 실습 7은 불러오기부터 판단 문장까지 전체 워크플로우를 두 데이터에 적용하는 종합 문제입니다. 오늘 배운 것이 전부 들어갑니다.

실습 7 보충답안 — 다섯 단계 전부

import pandas as pd

PATH = "data/13_diecasting_shot.csv"

① 불러오기 + ② 확인

df = pd.read_csv(PATH)

print(f"[구조] {df.shape} · 결측 {df.isna().sum().sum()}건")

③ 복합 조건으로 위험 설비 필터링

```
risky = (df["비스킷두께"] >= 13) & (df["사이클타임"] >= 25)
print(f"[위험] {risky.sum()}건 ({risky.mean()*100:.1f}%)")
```

④ 비스킷두께 내림차순 정렬

```
ranked = df[risky].sort_values("비스킷두께", ascending=False, kind="stable")
```

⑤ 주요 열만 선택

```
report = ranked.head(5)[["샷", "비스킷두께", "사이클타임", "품질등급"]]
print(report.to_string(index=False))
```

판단 문장

```
worst = ranked.iloc[0]
print(f"\n가장 위험한 설비는 {worst['샷']}번 샷입니다. "
      f"비스킷두께 {worst['비스킷두께']}, 사이클타임 {worst['사이클타임']}로 "
      f"품질등급은 {worst['품질등급']}입니다.")
```

▶ [구조] (200, 7) · 결측 70건

▶ [위험] 83건 (41.5%)

▶ 샷 비스킷두께 사이클타임 품질등급

▶ 198 27.0 36.6 불량

▶ 192 26.0 36.9 불량

▶ 171 20.0 36.1 주의

▶ 197 20.0 36.2 불량

▶ 121 19.0 33.9 주의

▶

▶ 가장 위험한 설비는 198번 샷입니다. 비스킷두께 27.0, 사이클타임 36.6으로 품질등급은 불량입니다.

실습 7 — 두 번째 흐름

같은 흐름을 불량 데이터에만 적용해 비교

```
defective = df[df["품질등급"] == "불량"]
```

```
top = defective.sort_values("비스킷두께", ascending=False, kind="stable").head(5)
```

```
print(f"불량 {len(defective)}건 중 두께 상위 5건")
```

```
print(top[["샷", "비스킷두께"]].to_string(index=False))
```

```
print(f"불량 평균 두께: {defective['비스킷두께'].mean():.1f}")
```

```
print(f"전체 평균 두께: {df['비스킷두께'].mean():.1f}")
```

▶ 불량 20건 중 두께 상위 5건

▶ 샷 비스킷두께

▶ 198 27.0

▶ 192 26.0

▶ 197 20.0

▶ 193 19.0

▶ 194 19.0

▶ 불량 평균 두께: 15.0

▶ 전체 평균 두께: 14.2

설비 적용

불량품의 평균 비스킷두께(15.0)가 전체 평균(14.2)보다 **0.8 높습니다**. 그리고 불량 중 두께 상위 5건이 전부 190번대 샷입니다. **비스킷두께가 두꺼워지는 것이 불량**의 **선행 신호**일 가능성이 보입니다. 다음 주에 배울 그룹 집계와 상관관계로 이 가설을 수치로 확인하게 됩니다.

C-3. 다음 시간 예고 — 14_01 빈도와 그룹 집계

오늘로 **Pandas 데이터 선택·필터링 파트**가 끝났습니다. 교안 14_01~14_03이 이미 배포되어 있고, 다음 시간부터 집계로 넘어갑니다.

교안	주제	핵심 표현	오늘과의 연결
14_01	빈도와 그룹 집계	<code>value_counts()</code> · <code>pd.cut()</code>	오늘 쓴 <code>value_counts()</code> 의 정식 학습
14_01	비율과 불균형 데이터	<code>normalize=True</code>	오늘의 조건 <code>.mean()</code> 과 같은 발상
14_02	그룹별 통계량	<code>groupby()</code>	"불량만" 대신 "등급별로 한 번에"
14_03	상관관계와 통합 리포트	<code>corr()</code>	두 지표가 함께 움직이는지

교안	주제	핵심 표현	오늘과의 연결
			수치로

연결

오늘 **"불량만 골라 평균을 낸다"** 를 조건 필터링으로 했습니다. 다음 시간에 배울 groupby는 **"양품·주의·불량 각각의 평균을 한 번에"** 냅니다. 조건을 세 번 거는 대신 한 줄로 끝나는 것이고, 오늘 것을 이해했다면 왜 편한지 바로 체감됩니다.

개념

교안 13_02 마지막 문장이 파트를 이렇게 닫습니다 — **"두 데이터는 내용이 다른데 왜 같은 방법으로 분석되는가. 행과 열을 고르고, 조건으로 거르고, 정렬하는 기본기는 동일하기 때문입니다."** 설비 로그든 매출 데이터든 이 기본기 위에 모든 분석이 쌓입니다.

CLOSING

맺음말

오늘의 성취와 복습 루틴

오늘 넘은 선 — 위치에서 조건으로

어제까지는 "몇 번째 행" 을 알아야 데이터를 꺼낼 수 있었습니다. 오늘은 "어떤 성질을 가진 행" 이면 위치를 몰라도 다 찾아낼 수 있게 됐습니다. 이것이 현장 질문에 답하는 방식입니다.

그리고 필터링과 정렬을 엮어 "가장 위험한 설비 5대" 라는 질문에 한 줄로 답했습니다. 도구는 두 개뿐인데 조합하니 분석이 됩니다. 교안의 말대로 **도구를 엮는 흐름이 분석의 진짜 힘**입니다.

오늘 할 수 있게 된 것	확인 질문	못 하면
조건을 Boolean Series로 만든다	<code>(s >= 13).sum()</code> 은 무엇을 세나?	STEP 1
조건으로 행을 추출한다	바깥 구조는 무엇으로 고정되나?	STEP 2
두 조건을 묶는다	<code>and</code> 를 쓰면 왜 오류가 나나?	STEP 3
부정·목록·범위 조건을 쓴다	<code>between(210, 230)</code> 은 경계를 포함하나?	STEP 4
결측이 조건에 미치는 영향을 안다	<code>~(A)</code> 와 부등호 뒤집기는 왜 다른가?	STEP 4-1-1
위험 순으로 정렬한다	<code>ascending=False</code> 는 무슨 뜻인가?	STEP 6
다중 키로 우선순위를 준다	리스트에 적은 순서가 뜻하는 것은?	STEP 7
메서드 체이닝으로 잇는다	필터→정렬→상위를 한 줄로 쓸 수 있나?	STEP 7
<code>copy</code> 경고의 의미를 안다	수정할 건데 경고가 뜨면 무엇을 붙이나?	STEP 8

복습 루틴 — 35분

오늘은 **문법이 적고 조합이 많습니다**. 도구 여섯 개(`&` | `~` `isin` `between` `contains`)를 각각 한 번씩 쳐 보고, 그다음 조합해 보는 순서가 좋습니다.



여섯 도구를 한 화면에

② 도구 여섯 개를 한 파일에서 확인

```
df = pd.read_csv("data/13_diecasting_shot.csv")
tests = [
   ("&", (df["비스킷두께"] >= 13) & (df["사이클타임"] >= 25)),
   ("|", (df["비스킷두께"] >= 13) | (df["사이클타임"] >= 25)),
   ("~", ~(df["품질등급"] == "불량")),
   ("isin", df["품질등급"].isin(["양품", "주의"])),
   ("between", df["실린더압력"].between(210, 230)),
   ("contains", df["품질등급"].str.contains("양")),
]

for name, cond in tests:
    print(f"{name:10s} {cond.sum():3d}건 ({cond.mean()*100:5.1f}%)")
```

```
▶ &          83건 ( 41.5%)
▶ |          104건 ( 52.0%)
▶ ~          180건 ( 90.0%)
▶ isin       180건 ( 90.0%)
▶ between    89건 ( 44.5%)
▶ contains   162건 ( 81.0%)
```

단계	무엇을	왜
① 부록 A 훑기 (5분)	A-1·A-2·A-3 세 표를 눈으로 넘긴다	조건·묵기·정렬을 머리에 배치
② 도구 6개 (10분)	위 코드를 실행해 개수·비율 확인	각각을 한 번씩 손으로 쳐 봐야 붙는다
③ 실습 7 (15분)	C-2 보충답안을 보지 않고 짜 본다	다섯 단계 흐름을 한 번에 체감
④ 부록 B 교정 (5분)	SyntaxError · isin 대상 바로잡기	제출본이 실행되게 만든다

설비 적용

취업 관점에서 오늘 내용은 **실무 면접에서 코드로 직접 물어보는 범위**입니다. "이 데이터에서 불량 중 두께 상위 5건을 뽑아 보세요" 같은 문제가 그대로 나옵니다. 부록 A-5의 risk_report 함수처럼 **필터→정렬→상위→열선택을 함수로 묶어** 레포에 올려 두면, 반복 작업을 자동화할 줄 안다는 신호가 됩니다.

다음 시간부터는 **집계**입니다. 오늘이 "불량만 골라 평균"이었다면, 다음은 "등급별 평균을 한 번에"입니다.